Chapter 8: Integrity Constraints

8.1 Entity, Domain & Referential Integrity Rules • Assertions • Triggers — Concept

0. What Are Integrity Constraints?

Integrity Constraints (ICs) are rules that a Database Management System (DBMS) enforces on data in relations (tables) to ensure the data is **accurate, consistent, and valid** at all times — during insertion, updation, and deletion operations.

Purpose of Integrity Constraints:

- Maintain **correctness** and **consistency** of data in the database.
- Prevent **accidental damage** to the database due to invalid data entry.
- Enforce **business rules** automatically at the database level (not just application level).
- Preserve meaningful **relationships** between tables.

The three classical integrity rules studied in the relational model are:

- **Entity Integrity Rule** — concerns the Primary Key.
- **Domain Integrity Rule** — concerns valid values of an attribute.
- **Referential Integrity Rule** — concerns the Foreign Key / relationship between tables.

★ EXAM POINT

- Integrity constraints are enforced by the DBMS itself, not by the user — this is what makes them reliable.
- The three core rules are collectively called **Relational Integrity Constraints**.

1. Entity Integrity Rule

Definition: The Entity Integrity Rule states that the **Primary Key** of a table **cannot contain a NULL value**, and every row must have a **unique** value for the primary key.

Rule (formal statement):

No attribute participating in the Primary Key of a base relation is allowed to accept NULL values.

Why it matters: The primary key is used to uniquely identify each tuple (row). If it were NULL, that row could never be uniquely identified or referenced.

Example Table — STUDENT

RollNo (PK)	Name	Class
1	Amit	X
2	Sneha	X
NULL ■	Rahul	X

Row 3 violates Entity Integrity because RollNo (Primary Key) is NULL.

★ EXAM POINT

- Entity Integrity applies ONLY to the Primary Key, not to other candidate keys or normal attributes.
- Composite primary key: NO component attribute of the key may be NULL.

✔ REMEMBER

- Rule in one line: “Primary Key ≠ NULL, and Primary Key must be unique.”

✘ COMMON MISTAKE

- Students often think Entity Integrity means 'no NULLs anywhere in the table' — wrong! It applies only to the Primary Key column(s).

2. Domain Integrity Rule

Definition: The Domain Integrity Rule (also called **Domain Constraint**) states that the value of every attribute must be an **atomic value from its defined domain** — i.e., it must match the data type, format, and range specified for that column.

A “domain” defines:

- The **data type** (e.g., integer, varchar, date).
- The **size/length** allowed.
- The **range** or **set of permissible values** (e.g., Age between 18 and 60).
- Whether **NULL** is allowed for that attribute.

Example Table — EMPLOYEE (Domain of *Age*: Integer, 18–60)

EmpId	Name	Age
101	Priya	29
102	Rohan	45
103	Kabir	150 ■ (out of domain)

Row 3 violates Domain Integrity: 150 is not a valid Age value in the defined domain.

★ EXAM POINT

• Domain Integrity ensures each column only accepts values that are meaningful for that attribute (correct type + valid range).

• It is the most basic and most frequently enforced constraint (via data type + CHECK constraint concept).

✔ REMEMBER

• Rule in one line: “Every attribute value must belong to its own defined domain.”

✘ COMMON MISTAKE

• Do not confuse Domain Integrity with Referential Integrity — Domain checks a single column's own valid values; it does NOT involve another table.

3. Referential Integrity Rule

Definition: The Referential Integrity Rule states that a **Foreign Key** value in the child (referencing) table must either be **NULL**, or must **match an existing Primary Key value** in the parent (referenced) table.

Rule (formal statement):

If a relation R1 references a relation R2 through a Foreign Key, then every Foreign Key value in R1 must match a Primary Key value existing in R2, or be wholly NULL.

Diagram — Parent/Child Relationship

RollNo (PK)	Name	DeptNo (FK)
1	Amit	D01
2	Sneha	D02
3	Rahul	D05 ■

→ FK refers to PK

DeptNo (PK)	DeptName
D01	Computer Sc.
D02	Electronics
D03	Mechanical

STUDENT.DeptNo (FK) must exist in DEPARTMENT.DeptNo (PK). Row RollNo=3 (D05) violates Referential Integrity since D05 is not in DEPARTMENT.

Actions that Preserve Referential Integrity (Concept):

- **Restrict:** Disallow the delete/update on parent if child rows reference it.
- **Cascade:** Automatically delete/update matching child rows.
- **Set NULL:** Set the foreign key of child rows to NULL.

★ EXAM POINT

- Referential Integrity always involves TWO tables — a Parent table (with PK) and a Child table (with FK).
- A Foreign Key value CAN be NULL (if participation is optional) — this does not violate Referential Integrity.

✔ REMEMBER

- Rule in one line: “Every Foreign Key value must match a Primary Key value in the parent table, or be NULL.”

✘ COMMON MISTAKE

- A common exam trap: students think FK must always match a value. Remember — FK is allowed to be NULL unless a NOT NULL constraint is separately applied.

4. Comparison: Entity vs Domain vs Referential Integrity

Basis	Entity Integrity	Domain Integrity	Referential Integrity
Applies to	Primary Key	Any attribute (column)	Foreign Key
No. of tables involved	One table	One table	Two tables (Parent & Child)
NULL allowed?	Never (for PK)	Depends on column definition	Yes, FK may be NULL
Main concern	Uniqueness & identification of rows	Valid data type / range / format	Valid relationship / link between tables
Violation example	PK value repeated or NULL	Age = 500, Gender = 'Z'	FK value with no matching PK in parent

■ SHORTCUT

• Memory Trick — **E-D-R**: **E**ntity = own row identity, **D**omain = own column values, **R**eferential = relation to another table.

5. Assertions — Concept

Definition: An **Assertion** is a general-purpose, named integrity constraint written as a separate database object, used to enforce a condition that must **always hold true** for the database — often across **multiple tables** — and is not tied to any single INSERT/UPDATE/DELETE event.

Purpose:

- Enforce complex business rules that cannot be expressed using simple key or CHECK constraints.
- Enforce a condition **spanning (covering or extending) multiple tables** at once (e.g., total marks in one table must not exceed a limit defined in another).
- Constraint is checked automatically by the DBMS whenever any relevant table is modified.

Concept / Syntax (SQL Standard form):

```
CREATE ASSERTION assertion_name
CHECK ( condition );
```

Working: Once created, the DBMS evaluates the CHECK condition automatically every time a relevant table changes. If the condition becomes FALSE, the operation that caused the violation is **rejected**.

Simple Example (concept):

Rule: “The number of students enrolled in any course must never exceed the number of seats available for that course.” This rule compares data from an ENROLLMENT table and a COURSE table — an ideal case for an Assertion.

```
CREATE ASSERTION seat_limit_check
CHECK (
NOT EXISTS (
SELECT course_id FROM Enrollment
GROUP BY course_id
HAVING COUNT(*) > (SELECT seats FROM Course
WHERE Course.course_id = Enrollment.course_id)
)
);
```

★ EXAM POINT

• An Assertion is NOT attached to a specific table event — it is a standalone, global constraint.

• Assertions can reference multiple relations in a single condition; simple CHECK constraints (on a column) cannot.

✔ REMEMBER

• Assertion = Independent rule for the whole database, automatically enforced, no explicit trigger event needed.

6. Triggers — Concept

Definition: A **Trigger** is a stored procedure that is **automatically executed (fired)** by the DBMS in response to a specific **event** — INSERT, UPDATE, or DELETE — occurring on a specified table.

Purpose:

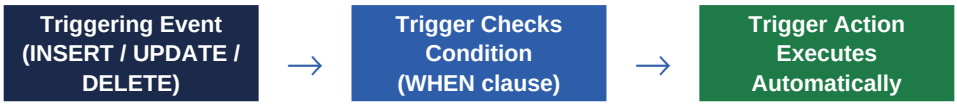
- Automatically enforce business rules when data changes.
- Maintain derived/summary data automatically (e.g., updating stock after a sale).
- Audit or log changes made to a table.
- Prevent invalid operations beyond what standard constraints can check.

Triggering Event: A trigger is always associated with one (or more) of:

- **INSERT** — fires when a new row is added.
- **UPDATE** — fires when an existing row is modified.
- **DELETE** — fires when a row is removed.

Triggers can also fire **BEFORE** or **AFTER** the triggering event.

Basic Working / Execution Flow:



A trigger 'wakes up' automatically when its defined event occurs, checks the condition, then performs the action — with no manual call needed.

Concept / Syntax:

```
CREATE TRIGGER trigger_name
BEFORE / AFTER INSERT / UPDATE / DELETE ON table_name
FOR EACH ROW
BEGIN
-- action(s) to be performed
END;
```

Simple Example (concept):

Rule: “Whenever a new employee's salary is updated, automatically log the old and new salary into an AUDIT table.”

```
CREATE TRIGGER salary_audit
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
INSERT INTO Audit (emp_id, old_salary, new_salary)
VALUES (OLD.emp_id, OLD.salary, NEW.salary);
END;
```

★ EXAM POINT

• A trigger is always tied to a specific event (INSERT/UPDATE/DELETE) on a specific table — unlike an Assertion.

• Triggers can use OLD and NEW row values to compare data before and after the change.

✔ REMEMBER

• Trigger = 'IF this event happens on this table, THEN automatically do this action.'

✘ COMMON MISTAKE

• Don't confuse a Trigger with a stored procedure call — a trigger fires automatically; it is never called explicitly by the user.

7. Assertions vs Triggers

Basis	Assertion	Trigger
Nature	Passive, declarative constraint	Active, procedural mechanism
Invocation	Automatically checked on any relevant change; not tied to one event	Fires only on a specific event: INSERT / UPDATE / DELETE
Scope	Can span (cover or extend across) multiple tables in one condition	Defined on a single specified table
Action on violation	Rejects the operation if condition becomes false	Executes a defined action/procedure (may or may not block the operation)
Complexity of logic	Limited to a boolean CHECK condition	Can contain full procedural logic (multiple statements)
Primary use	Enforcing global business rules / cross-table constraints	Automating actions, auditing, cascading updates, complex validation

■ SHORTCUT

- Memory Trick: **Assertion = Always-on Rule (passive)**; **Trigger = Task on Trigger-event (active)**.

8. Quick Revision / Cheat Sheet

Core Definitions (one-liners):

• **Integrity Constraint:** A rule enforced by the DBMS to keep data accurate and consistent.

• **Entity Integrity:** Primary Key must never be NULL and must be unique.

• **Domain Integrity:** Every attribute value must belong to its defined domain (type/range).

• **Referential Integrity:** Foreign Key must match an existing Primary Key in the parent table, or be NULL.

• **Assertion:** A standalone, always-checked constraint, often spanning (covering or extending) multiple tables.

• **Trigger:** A stored procedure that fires automatically on INSERT/UPDATE/DELETE.

Important One-Line Exam Facts:

✓ Entity Integrity Rule applies ONLY to the Primary Key.

✓ Domain Integrity is checked using data type, size, and range (concept behind CHECK constraint).

✓ Referential Integrity always involves a Parent (PK) table and a Child (FK) table.

✓ A Foreign Key CAN be NULL — it does not violate Referential Integrity.

✓ Assertion = passive rule, checked automatically, no explicit event required.

✓ Trigger = active rule, fires only on a specific event (INSERT/UPDATE/DELETE).

✓ Triggers can run BEFORE or AFTER their triggering event.

✓ OLD and NEW keywords in a trigger refer to the row's value before and after the change.

✓ 'CREATE ASSERTION ... CHECK(...)' is the standard SQL syntax pattern for assertions.

✓ 'CREATE TRIGGER ... BEFORE/AFTER event ON table' is the standard syntax pattern for triggers.

Final Comparison Snapshot:

Rule	Key Object	Tables Involved
Entity Integrity	Primary Key	1
Domain Integrity	Attribute/Column	1
Referential Integrity	Foreign Key ↔ Primary Key	2
Assertion	CHECK condition (named)	1 or more
Trigger	Event + Action (stored procedure)	1 (defined table)

End of Chapter 8 (Section 8.1) Study Notes — Entity, Domain & Referential Integrity Rules; Assertions and Triggers Concept.